



HANDLEIDING PRODUCTIE SV-BEVRAGING

Jorre Vannieuwenhuyze 

12 mei 2026 

1	Praktische informatie	3
2	Datamodel	4
2.1	Codelijsten	5
2.1.1	Algemene codelijsten	5
2.1.2	Specifiekere codelijsten	7
2.1.3	Verdere opmerkingen	8
2.2	Data structure definitions	9
3	Populatie-informatie	11
4	Dataverwerking voor de SV-bevraging	12
4.1	Editie-specifieke verwerking	12
4.1.1	Metadata	12
4.1.2	Aanmaak codeboek	15
4.1.3	Cleaning van de data	17
4.1.4	Aanmaak van de VOS'en	20
4.2	Klaarzetten van tabellen voor de cijferpagina's	22
4.3	Creatie van handleiding en technische nota	24
4.4	Creatie van Public en Scientific Use Files	25
5	Nieuwe vragen	26

De cleaning van de SV-bevragingen is work-in-progress. De ideeën en de opzet die ik hieronder voorstok zijn nog steeds slechts een aanzet om de zaken efficiënter maar vooral consistenten en minder fout-gevoelig te publiceren. De SV-bevragingen kunnen daarbij gebruikt worden als voorbeeld voor andere dataverwerkingsprocessen. Zo'n proces opzetten is sowieso een proces van trial and error, uittesten wat goed werkt en wat niet, steeds kijken welke mankementen nog niet zijn opgelost, en op die manier te evolueren naar een werkbare aanpak.

1. PRAKTISCHE INFORMATIE

- Alle bestanden bevinden zich in de map

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/
021_SVbevraging
```

Deze map is onderverdeeld in drie submappen:

- 01_code: bevat alle verwerkingscode voor de SV-bevraging. Deze map kan gesynchroniseerd worden met GitHub.

Opmerking 1: naast scripts bevat deze map ook enkele datasets die manueel worden aangemaakt om het productieproces overzichtelijk te houden. Deze werkwijze is momenteel nog niet standaard binnen de VSA.

Opmerking 2: de code omvat ook processtappen die idealiter door aparte, gespecialiseerde teams worden beheerd. Het gaat daarbij onder meer om de ontwikkeling van algemene programmeerfuncties, het beheer van het datamodel en de aggregatie van demografische gegevens voor de berekening van de gewichten.

- 02_sourcedata: opslagplaats voor alle aangeleverde ruwe datasets, georganiseerd per editie (SV0001, SV0002, SV0003, ...). Elke editie bevat twee submappen:

- + 01_sample: steekproefdata van Statbel
- + 02_survey: enquêtedata van de veldwerkbureaus

Wanneer er meerdere leveringen binnen één editie zijn, wordt aanbevolen om deze verder op te delen in mappen per aanleverdatum met mapnaam in het formaat YYYYMMDD.

Daarnaast bevat deze map ook de submap Demobel-stock, met bevolkingsdata die gebruikt wordt voor de berekening van populatiestatistieken en weegfactoren. Op termijn hoort deze data structureel thuis in:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_  
VSA/011_Demografie
```

waardoor de map Demobel-stock overbodig wordt.

- 03_deriveddata: bevat alle tussentijdse en afgeleide bestanden die uitsluitend intern worden gebruikt tijdens de verwerking.
- Gedeelde data wordt weggeschreven naar twee afzonderlijke mappen:
 - //WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/998_Scientific_Use_FilesBestanden voor intern gebruik binnen de VSA. Deze zijn toegankelijk voor alle VSA-medewerkers, maar niet bedoeld

voor externe publicatie. Voorbeelden zijn de Scientific Use Files (SUF's).

- `//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_Public_Use_Files`

Bestanden die publiek mogen worden gepubliceerd en ook extern toegankelijk zijn. Hieronder vallen onder meer de Public Use Files (PUF's) en de bestanden met de officiële Vlaamse openbare statistieken (VOS).

- Mappen, variabelen, codelijsten en andere technische objecten worden standaard in het Engels benoemd. Dit verhoogt de consistentie en bevordert hergebruik en duurzaamheid.
- De meeste databestanden worden momenteel opgeslagen in `.xlsx`-formaat. Dat is binnen de VSA het meest toegankelijke formaat voor inspectie en manuele aanpassingen, maar het is niet de meest efficiënte keuze. Formaten zoals `.csv`, `.parquet` of `.json` zijn performanter en robuuster en kunnen op termijn worden overwogen.
- Alle scripts zijn geschreven in R. Indien nodig kunnen deze in principe relatief eenvoudig worden omgezet naar Python.
- De map `01_code/00_functions` bevat een verzameling functies volgens de structuur van een R-package. De functies bevinden zich in de submap R en zijn gedocumenteerd via Roxygen2 in de code zelf.

2. DATAMODEL

De eerste stap in het verwerkingsproces is de opzet van een datamodel. Daarbij worden eerst codelijsten gedefinieerd, waarna data structure definitions (DSD's) worden opgesteld voor de Internal, Scientific en Public Use Files. In deze DSD's wordt een overzicht gegeven van de kolommen of variabelen, en worden codelijsten aan deze kolommen gekoppeld indien relevant. Deze opzet sluit aan bij de principes van SDMX.

Het datamodel wordt opgebouwd in de map `01_code/01_data_model`. Afgeleide codelijsten en data structure definitions worden weggeschreven naar `03_deriveddata/01_codelists` en `03_deriveddata/02_DSD`. Op langere termijn wordt het beheer van het datamodel idealiter gecentraliseerd bij een dedicated datamodelteam dat werkt in:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/002_DataModel
```

In principe hoeft het datamodel enkel te worden aangepast wanneer:

- een nieuwe vraag of antwoordcategorie wordt toegevoegd aan de SV-bevraging;
- een bestaande definitie of codering wordt gecorrigeerd;
- bijkomende validatieregels nodig zijn voor de verwerking.

2.1. CODELIJSTEN

De map `01_data_model/01_code_lists` bevat de scripts en invoerbestanden voor de aanmaak van de codelijsten. Deze codelijsten kunnen worden onderverdeeld in algemene codelijsten en codelijsten met een meer specifieke verwerking.

2.1.1. Algemene codelijsten

De map `01_general_codelists` bevat algemene codelijsten die geen specifieke behandeling vragen. De opzet ervan vraagt wel enige toelichting, omdat veel codelijsten inhoudelijk met elkaar samenhangen.

Een eerste voorbeeld is dat verschillende vragen in de SV-bevraging dezelfde niet-substantiële antwoordcategorieën gebruiken, zoals “Niet van toepassing” of “Ik weet het niet / geen antwoord”. Om consistentie te garanderen, worden deze categorieën centraal beheerd zodat hun labels overal identiek blijven.

Een tweede voorbeeld is dat sommige codelijsten een subset vormen van andere codelijsten. Zo kan een codelijst met de categorieën “man” en “vrouw” een subset zijn van een ruimere codelijst voor geslacht zoals die in de vragenlijst wordt gebruikt, bijvoorbeeld met categorieën “man”, “vrouw” en “X”. Ook in dat soort gevallen is centrale afstemming wenselijk om inconsistenties te vermijden.

De algemene codelijsten worden opgebouwd op basis van twee invoerbestanden:

- `input/code_list_values.xlsx`: bevat een overzicht van alle mogelijke waarden die op codelijsten kunnen voorkomen. Dit bestand bevat onder meer:
 - `value`: een gestandaardiseerde code die wordt gebruikt in datasets en scripts;
 - `description_nl`: een inhoudelijke beschrijving van de waarde in het Nederlands, louter bedoeld als naslag;

- `label_nl`, `label_fr`, `label_en`, ...: labels in verschillende talen die gebruikt worden in output zoals public use files en gepubliceerde tabellen.

Door deze labels centraal te beheren, kunnen wijzigingen eenvoudig en consistent worden doorgevoerd. Conceptueel sluit dit bestand aan bij het idee van een conceptenlijst voor codelijstwaarden.

- `input/code_list_skeletons.xlsx`: bevat de structuur van de codelijsten. Elk tabblad beschrijft een reeks codelijsten die inhoudelijk samenhangen of subsets van elkaar vormen. Dit bestand bevat kolommen:
 - `value`: sleutel die verwijst naar `code_list_values.xlsx`;
 - `valuen`: numerieke code die een consistente ordening van categorieën mogelijk maakt;
 - één of meerdere kolommen met aan te maken codelijsten. De kolomnaam is de naam van de codelijst. Waarden `TRUE` geven aan dat een waarde moet worden opgenomen in de codelijst; `FALSE` betekent dat de waarde niet wordt opgenomen.

Het script `create_general_code_lists.R` combineert beide bestanden en schrijft de codelijsten weg naar `03_deriveddata/01_codelists`. Daar wordt elke codelijst als een afzonderlijk `.parquet`-bestand (voor verder gebruikt) en `.xlsx`-bestand (als naslagwerk) bewaard.

Elke codelijst bevat minstens:

- `value` en eventueel `valuen`;
- labels in verschillende talen;
- indien relevant: mappings naar bredere of beperktere codelijsten die als subsetrelaties kunnen worden gebruikt tijdens de dataverwerking.

Die mappings maken het bijvoorbeeld mogelijk om variabelen met niet-substantiële categorieën op een consistente manier om te zetten naar een versie zonder zulke niet-substantiële categorieën.

2.1.2. Specifieke codelijsten

Naast de algemene codelijsten bevat het datamodel ook een aantal codelijsten met een specifiekere verwerking.

NIS-codes

Het script `02_NIS_codes/create_code_lists_NIScodes.R` maakt code-lijsten aan voor NIS-codes van gewesten, provincies, arrondissementen, gemeenten en deelgemeenten. Hiervoor worden de Statbel-bestanden gebruikt die kunnen worden gedownload via:

```
https://statbel.fgov.be/nl/over-statbel/  
methodologie/classificaties/geografie
```

De bronbestanden worden bewaard in `02_NIS_codes/input`.

Er worden versies aangemaakt voor de jaren 1983, 1995, 2019 en 2025, omdat in die jaren wijzigingen in de NIS-codering werden doorgevoerd. Voor elke versie worden codelijsten gemaakt op verschillende niveaus:

- 0 = het rijk
- 1 = gewesten
- 2 = provincies
- 3 = arrondissementen
- 4 = gemeenten
- 5 = deelgemeenten

In de map `03_deriveddata/01_codelists` vind je deze bestanden terug als `cl_refnisVERSIONlvlNIVEAU.xlsx`. Daarnaast wordt per versie ook een algemene codelijst aangemaakt, `cl_refnisVERSION.xlsx`, waarin de verschillende niveaus gecombineerd worden.

Deze codelijsten bevatten ook mappings tussen niveaus en versies, zodat bijvoorbeeld gemeenten kunnen worden gekoppeld aan provincies of oudere coderingen aan nieuwere.

Landencodes

Landen worden op verschillende manieren gecodeerd en gegroepeerd. Het script `03_country_codes/create_code_lists_countries.R` maakt hiervoor codelijsten aan. Het gaat onder meer om:

- ISO2-codes;
- ISO3-codes;
- NIS-landencodes;
- indelingen naar landengroepen.

2.1.3. Verdere opmerkingen

Andere codelijsten

Voor de NIS-codes en landencodes maken de scripts heel wat codelijsten aan die niet worden gebruikt voor de verwerking van de SV-bevraging. De SV-bevraging heeft enkel NIS-codes op gemeenteniveau nodig die enkel naar provincieniveau worden omgezet. Voor landencodes wordt binnen de SV-bevraging enkel de NIS-landencodes gebruikt.

Waarom maken de scripts dan ook andere codelijsten aan? Een deel van deze scripts werd oorspronkelijk ontwikkeld in het kader van een breder transitietraject binnen de VSA. Andere dataverwerkingsprocessen kunnen gebruik maken van die andere codelijsten. Bovendien zijn er ook al scripts beschikbaar die momenteel niet rechtstreeks worden gebruikt voor de SV-bevraging, maar later nuttig kunnen zijn voor andere dataverwerkingsprocessen. Deze bevinden zich in:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/002_  
DataModel/00_Code
```

Het gaat onder meer om voorbereidende scripts voor codelijsten van:

- statistische sectoren;
- tijdsvariabelen;
- postcodes;
- andere variabelen uit de Demobel-stand van de bevolking.

Clean run

Het script `create_code_lists.R` maakt alle codelijsten in `03_deriveddata/01_codelists` opnieuw aan vanaf een lege toestand. Het script bevat zelf geen logica om codelijsten op te bouwen, maar roept de onderliggende scripts in de submappen aan. Vooraf wordt de doelmap verwijderd en opnieuw aangemaakt. Hierdoor vermijdt je dat verouderde bestanden blijven staan en voer je een clean run uit.

Wat kan nog beter?

De scripts kunnen nog verder worden uitgebreid met bijkomende kwaliteitscontroles, bijvoorbeeld:

- controle op unieke namen van codelijsten;
- controle op verplichte kolommen zoals `value` en `label_nl`;
- controle op unieke `value` en `valuen` binnen een codelijst;
- controle op consistente mappings tussen gerelateerde codelijsten;
- de aanmaak van de algemene codelijsten kan waarschijnlijk nog efficiënter. Wanneer dit wordt uitgebreid naar andere dataprocessen

binnen de VSA, lijkt het ook overzichtelijker wanneer deze thematisch worden beheerd.

2.2. DATA STRUCTURE DEFINITIONS

Nadat de codelijsten zijn aangemaakt, kan de data structure definition (DSD) worden opgesteld. In de DSD wordt vastgelegd:

- welke variabelen kunnen voorkomen in een dataset;
- welk type deze variabelen hebben;
- welke validatieregels erop van toepassing zijn;
- en, indien relevant, aan welke codelijst zij gekoppeld zijn.

Binnen het datamodel worden variabelen beschouwd als concepten in de zin van SDMX. De DSD vormt daarmee de inhoudelijke en technische basis voor de verdere dataverwerking en voor de sterschema's in de database.

De aanpak voor de creatie van een DSD bestaat uit twee stappen:

1. eerst wordt één centrale DSD opgesteld met alle mogelijke variabelen en hun algemene criteria;
2. vervolgens worden voor specifieke outputbestanden enkel de relevante concepten geselecteerd en eventueel verder verfijnd.

De aanmaak van de algemene DSD start vanuit het bestand:

`01_code/01_data_model/02_DSD/dsd_variables.xlsx`

Dit bestand bevat een overzicht van alle mogelijke variabelen en bevat onder meer volgende kolommen:

- `concept_id`: unieke identificatienaam van de variabele;
- `description_nl`: inhoudelijke beschrijving in het Nederlands. Deze wordt verder niet gebruikt en kan dus heel flexibel worden gebruikt als naslagwerk;
- `label_nl`, `label_fr`, `label_en`: korte labels in verschillende talen die worden overgenomen in data-ontsluiting;
- `constraint_type`: opslagformaat of datatype, met mogelijke waarden `character`, `codelist`, `integer`, `numeric`, `date` of `datetime`;
- `constraint_codelist`: verwijzing naar de gebruikte codelijst, indien van toepassing;
- `constraint_regex`: reguliere expressie waaraan karakteriële waarden moeten voldoen;
- `constraint_min`: minimale toegelaten waarde voor numerieke variabelen;
- `constraint_max`: maximale toegelaten waarde voor numerieke variabelen;

- `constraint_allow_missings`: logische indicator die aangeeft of missende waarden toegelaten zijn;

Het script `01_code/01_data_model/02_DSD/check_and_publish_dsd.R` kopieert `dsd_variables.xlsx` naar `03_deriveddata/02_DSD` als `.parquet`-bestand (voor verder gebruik) en `.xlsx`-bestand (als naslagwerk), maar voert eerst verschillende controles uit. Voorbeelden zijn:

- controle op unieke `concept_id`'s;
- controle of gebruikte codelijsten effectief bestaan in `03_deriveddata/01_codelists`;
- controle of regular expressions geldig zijn;
- controle of min en max logisch zijn ingevuld;
- controle op consistentie tussen datatype en validatieregels.

Het gepubliceerde bestand `03_deriveddata/02_DSD/dsd.xlsx` is vervolgens de formele basis voor de verdere dataverwerking. De centrale DSD bevat algemene definities en validatieregels. Belangrijk is dat:

- de centrale DSD steeds de referentie blijft;
- afwijkingen of verfijningen expliciet worden gedocumenteerd in code of metadata;
- nieuwe concepten of structurele wijzigingen zoveel mogelijk eerst in het datamodel worden opgenomen, en niet ad hoc in een afzonderlijk script.

Wat kan beter?

- Er kan een bijkomende constraint `complete` worden toegevoegd, een indicator die aangeeft of in een dataset alle mogelijke waarden verwacht worden voor deze variabele. Dit is vooral relevant voor tabellen waarin de volledige combinatie van categorieën aanwezig moet zijn. Bijvoorbeeld: als voor elke gemeente een waarde moet voorkomen, moet `complete = TRUE` staan voor de betrokken gemeentecodevariabele.
- Mijn idee was dat analisten tijdens verdere verwerking de regels uit de DSD verder kunnen verfijnen voor een specifieke outputdataset, zolang dit gebeurt op een consistente en gedocumenteerde manier. Zo kan een variabele in de centrale DSD breder gedefinieerd zijn dan in een specifieke public use file of analysetabel. Zo kan de algemene DSD opleggen dat de minimumwaarde voor geboortjaar 1900 is maar kan dit verder worden aangepast naar bijvoorbeeld 1920, zolang de nieuwe minimumwaarde maar groter is dan de algemene minimumwaarde.

- Op langere termijn kan het datamodel nog verder worden uitgebouwd door:
 - conceptenlijsten voor variabelen en codelijst- waarden nauwer op elkaar af te stemmen. Als je een tabel pivoteert worden variabelen immers waarden op één variabele of vice versa;
 - outputtabellen ook formeel te koppelen aan het datamodel;
 - validatieregels verder te standaardiseren over verwerkingsprocessen heen.

3. POPULATIE-INFORMATIE

De tweede stap in het verwerkingsproces is de aggregatie van bevolkingsstatistieken voor de berekening van de poststratificatiegewichten in de SV-bevraging. Hiervoor wordt gebruikgemaakt van stockbevolkingsdata.

De aggregatie gebeurt per jaar en niet per editie van de SV-bevraging, omdat de bevolkingsdata ook per jaar wordt aangeleverd. Daarom vormt dit een afzonderlijke stap in het proces.

De brondata wordt bewaard in:

```
021_SVbevraging/02_sourcedata/Demobel-stock
```

Steeds in .sav-formaat. Vraag deze databestanden bij voorkeur op bij de demografen. Gebruik daarbij consequent de bestandsnaam `stock_YYYY.sav`, omdat de scripts deze bestanden automatisch ophalen. Bij afwijkende bestandsnamen zullen de scripts niet correct werken en een foutmelding geven.

De aggregatie gebeurt via de scripts in de map `01_code/02_population_data`. Er is doorgaans één script per jaar, maar deze scripts roepen telkens enkel de functie `SVsurvey_aggregate_population()` aan uit `01_code/00_functions/R`. Die functie schrijft de geaggregeerde cijfers automatisch weg naar:

```
03_deriveddata/03_aggregated_population_data
```

Wat kan beter?

Inhoudelijk is dit in feite een proces dat idealiter structureel zou worden opgenomen binnen de demografische dataprocessen, bijvoorbeeld door de demografen of de data engineers die verantwoordelijk zijn voor de verwerking in:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/011_
Demografie
```

In dat geval zouden de scripts voor de SV-bevraging de geaggregeerde data daar rechtstreeks kunnen ophalen. Zo wordt dubbele databewaring en -verwerking vermeden.

4. DATAVERWERKING VOOR DE SV-BEVRAGING

De volgende stap is de verwerking van de data per editie van de SV-bevraging. Deze verwerking gebeurt in de map `01_code/03_process_editions`. Daarin vind je een scriptmap per editie, aangevuld met een map voor algemene metadata.

4.1. EDITIE-SPECIFIEKE VERWERKING

De verwerking van een editie bestaat uit verschillende opeenvolgende stappen:

1. metadata beheren;
2. codeboek aanmaken;
3. ruwe data valideren en cleanen;
4. analysedata en VOS'en aanmaken.

4.1.1. Metadata

Metadata wordt zo veel mogelijk overzichtelijk bewaard in databestanden en niet in programmascripts. Die bestanden moeten actief worden aangevuld bij elke editie. Als noodzakelijke informatie ontbreekt, geven de scripts foutmeldingen zodat duidelijk is wat nog moet worden ingevuld.

De belangrijkste metadata-bestanden zijn:

- `01_code/03_process_editions/Metadata/Editions.xlsx`
Bevat een overzicht van de verschillende edities van de SV-bevraging.
- `01_code/03_process_editions/Metadata/Mailings.xlsx`
Bevat een overzicht van alle contactpogingen per veldwerkperiode binnen de SV-bevraging.

- 01_code/03_process_editions/Metadata/Sports.xlsx
Bevat de lijst met sporten die gecodeerd worden door Tina. Aan deze lijst mogen in principe enkel nieuwe sporten worden toegevoegd. Er mag niets worden verwijderd. Het bestand koppelt een gestandaardiseerde beschrijving aan een numerieke code.
- 01_code/03_process_editions/Metadata/Questions.xlsx
Bevat een overzicht van alle vragen in de SV-bevraging. Dit bestand koppelt de concept_id's uit het datamodel aan concrete vraagverwoordingen. De vraagverwoording is terug te vinden in de kolommen question en question_item. Daarnaast bevat het bestand ook informatie over de vragenlijstmodule in kolom question_module.

Dit bestand kan worden gebruikt om beperkte tekstuele wijzigingen in vraagverwoordingen te registreren, voor zover daarbij geen belangrijke resultaatbreuk wordt verwacht. Daarvoor wordt de kolom from_surv_id gebruikt. Deze kolom geeft aan vanaf welke editie een bepaalde formulering wordt gebruikt.

Bijvoorbeeld: als in een latere editie de formulering van een bestaande vraag beperkt wijzigt, kunnen beide versies in Questions.xlsx worden opgenomen, elk met hun eigen starteditie (from_surv_id). Tijdens de verwerking wordt dan automatisch de relevante versie gekozen.

Belangrijk: wanneer een wijziging in vraagverwoording inhoudelijk te groot genoeg is en een breuk in de resultaten kan veroorzaken, moet in principe een nieuw concept worden gedefinieerd in het datamodel in plaats van een loutere wijziging in vraagverwoording.

Meer over het gebruik van dit bestand staat in paragraaf 4.1.2 en 4.1.3.

- 01_code/03_process_editions/Metadata/Answers.xlsx
Bevat alle antwoordverwoordingen die in de vragenlijst worden getoond en die in principe ook als value labels in de aangeleverde data voorkomen. De value label wordt gekoppeld aan de codelijsten via de unieke value.

Net zoals Questions.xlsx kan dit bestand worden gebruikt om beperkte tekstuele wijzigingen in antwoordverwoordingen te documenteren. Ook hier gebeurt dat via de kolom from_surv_id.

Bij de aanmaak van codeboeken en tijdens de cleaning wordt steeds de meest recente geldige antwoordverwoording gekozen, namelijk de versie met de hoogste from_surv_id die niet hoger is dan de editie die verwerkt wordt.

Ook hier geldt: bij een inhoudelijk belangrijke wijziging in de antwoordcategorieën of betekenis van de categorieën moet een nieuw concept of een nieuwe codelijst worden gedefinieerd in het datamodel.

Meer over het gebruik van dit bestand staat in paragraaf 4.1.2 en 4.1.3.

- 01_code/03_process_editions/Metadata/Variables_for_VOS.xlsx

Dit bestand bevat alle afgeleide variabelen die worden gebruikt in een tussentijdse analysedataset met het oog op de aanmaak van de openbare statistieken. Het gaat vooral om:

- gedichotomiseerde variabelen, meestal eindigend op `_dich`;
- variabelen waarbij niet-substantiële categorieën werden verwijderd, meestal eindigend op `_subst`;
- numerieke variabelen voor de berekening van gemiddelden, meestal eindigend op `_n`.

Meer over het gebruik van dit bestand staat in paragraaf 4.1.4. Het bestand bevat onder meer:

- `VOSn`: het numerieke nummer van de VOS waarvoor de variabele wordt gebruikt;
- `VOSname`: de naam van de VOS waarvoor de variabele wordt gebruikt;
- `varname`: de naam van de analysevariabele;
- `varlabel`: een label dat kan worden overgenomen in opgeleverde datasets;
- `calc_p_value`: een verouderde vlagvariabele die aangeeft of voor deze variabele *p*-waarden moesten worden berekend. Deze kolom wordt niet meer gebruikt;
- `type`: het type statistiek dat voor deze variabele moet worden berekend:
 - + `percentage`: voor dichotome logische variabelen (TRUE/FALSE);
 - + `multinom`: voor percentages per categorie;
 - + `mean`: voor gemiddelden van numerieke variabelen;
- `description`: een inhoudelijke beschrijving van de statistieken.

De kolomnamen van dit bestand kunnen op termijn nog verbeterd worden. Op langere termijn zou dit bestand mogelijk zelfs kunnen verdwijnen indien mappings en analysetypen rechtstreeks vanuit het datamodel en de codelijsten kunnen worden afgeleid.

Wat kan beter?

In het vorig werkproces was de meta-informatie meer geïntegreerd, vooral het bestand

//WV162699/fs_kb_dkb/svr/4_Beveiligde_data/SV-bevragingen/
01_Metadata/variabelen.xlsx

bevatte veel informatie. Dit bestand had echter ook wat nadelen:

- je kon moeilijk verschillende versies maken van een variabele (bv. als de vewoording licht verandert)
- het verschil tussen de steekproef- en de survey-data was onduidelijk en onhandig
- Het was moeilijk om de volgorde van de vragen ergens te bewaren per editie

Om al deze redenen heb ik nu dit bestand uit elkaar gehaald en de informatie bewaart in verschillende bestanden. Daarmee zijn bovenstaande problemen opgelost:

- Als de vraagverwoording verandert kan je dit documenteren in Questions.xlsx en Answers.xlsx
- het verschil tussen de steekproef- en de survey-data wordt gemaakt omdat er twee aparte bestanden zijn per editie, namelijk SVXXXX_variables_sample.xlsx en SVXXXX_variables_survey.xlsx (zie verder).
- Het bestand SVXXXX_variables_survey.xlsx bepaalt ook de volgorde van de vragen voor elke editie apart (zie verder).

Dit systeem is flexibeler, maar bij het uittesten leek het wel een minder gebruiksvriendelijk. Misschien is het dus beter de bestanden weer op één of andere manier samen te voegen, maar wel met de nieuwe ideeën erin.

4.1.2. Aanmaak codeboek

De eerste concrete stap in de verwerking van een nieuwe editie is de aanmaak van het codeboek. Dit codeboek wordt doorgestuurd naar het veldwerkbureau en beschrijft welke variabelen verwacht worden, hoe deze benoemd moeten worden en welke labels en antwoordcategorieën van toepassing zijn.

De werkwijze is als volgt:

1. Als vraag- of antwoordverwoordingen zijn aangepast – en het gaat enkel om beperkte tekstuele wijzigingen – werk dan de bestanden

```
01_code/03_process_editions/Metadata/Questions.  
xlsx
```

en

```
01_code/03_process_editions/Metadata/Answers.  
xlsx
```

bij.

In principe zouden zulke wijzigingen uitzonderlijk moeten zijn. Bij inhoudelijk belangrijke wijzigingen moet eerder een nieuw concept worden gedefinieerd in het datamodel, in plaats van een loutere aanpassing in vraagverwoording, zodat de antwoorden van de oude en nieuwe vragen in aparte variabelen worden bewaard.

2. Zorg ervoor dat het bestand

```
01_code/03_process_editions/SVXXXX/input/  
SVXXXX_variables_survey.xlsx
```

klaarstaat.

Dit bestand geeft een overzicht van alle variabelen die door het veldwerkbureau moeten worden opgeleverd. Het bevat onder meer:

- `variable`: de naam die verwacht wordt in de dataset die het veldwerkbureau zal opleveren;
- `concept_id`: het gestandaardiseerde concept waaraan deze variabele gekoppeld wordt.

De volgorde van de variabelen in dit bestand is belangrijk, omdat deze integraal wordt overgenomen in het codeboek. Voor elke variabele moet een geldig concept bestaan in de DSD. Zo niet zullen de scripts foutmeldingen geven.

3. Maak het script

```
01_code/03_process_editions/SVXXXX/01_create_  
codebook_SVXXXX.R
```

aan. Dit kan doorgaans door een script van een vorige editie te kopiëren en het editienummer aan te passen.

In dit script wordt de functie `SVsurvey_create_codebook()` aangeroepen. Deze functie:

- leest het bestand `SVXXXX_variables_survey.xlsx` in;

- koppelt de variabelen via `concept_id` aan de juiste vraagverwoording, variabelelabels, codelijsten en antwoordverwoordingen;
- gebruikt daarvoor de DSD, `Questions.xlsx` en `Answers.xlsx`;
- compileert het codeboek;
- en schrijft het resultaat weg naar:

```
03_deriveddata/04_Codebooks/Codebook_SVXXXX.
xlsx
```

4.1.3. Cleaning van de data

Nadat het codeboek is doorgestuurd en het veldwerkbureau de data heeft opgeleverd, kan de feitelijke gegevensverwerking starten. De eerste stap daarin is de validatie en cleaning van de aangeleverde steekproef- en surveydata.

De werkwijze is als volgt:

1. Zorg ervoor dat de steekproefdata klaarstaat in:

```
02_sourcedata/SVXXXX/01_sample
```

2. Zorg ervoor dat het bestand

```
01_code/03_process_editions/SVXXXX/input/
SVXXXX_variables_sample.xlsx
```

klaarstaat.

Dit bestand geeft een overzicht van de variabelen die uit de steekproefdata worden overgenomen. Het bevat minstens:

- `variable`: de naam van de variabele in de steekproefdata;
- `concept_id`: de gestandaardiseerde naam die in de gekuiste data gebruikt zal worden.

De namen in kolom `variable` moeten exact overeenkomen met de kolomnamen in de steekproefdata en zijn hoofdlettergevoelig. Enkel variabelen die relevant zijn voor de cleaning hoeven in dit bestand te worden opgenomen.

3. Zorg ervoor dat de surveydata klaarstaat in:

```
02_sourcedata/SVXXXX/02_survey
```

Als er meerdere leveringen zijn, is het aanbevolen om submappen aan te maken met de aanleverdatum als mapnaam in het formaat `YYYYMMDD`.

4. Zorg ervoor dat het bestand

```
01_code/03_process_editions/SVXXXX/input/  
SVXXXX_variables_survey.xlsx
```

klaarstaat.

Ook hier moeten de variabelenamen in kolom `variable` exact overeenkomen met de kolomnamen in de `surveydata`. De kolom `concept_id` verwijst opnieuw naar de gestandaardiseerde namen in het datamodel. Variabelen die niet gebruikt worden in de cleaning hoeven niet opgenomen te worden.

5. Maak het script

```
01_code/03_process_editions/SVXXXX/02_clean_  
data_SVXXXX.R
```

aan. Dit script bevat eerst eventuele manuele of editie-specifieke voorbereidingen, en daarna de structurele cleaning.

In de praktijk bevat dit script doorgaans volgende blokken:

a. Inladen en voorbereiden van de steekproefdata.

Zorg ervoor dat:

- enkel de variabelen behouden blijven die in `SVXXXX_variables_sample.xlsx` zijn opgelijst;
- deze variabelen de juiste namen krijgen;
- de waarden voldoen aan de criteria uit de DSD (bijvoorbeeld correcte types, geldige codelijstwaarden, geldige datums, ...).

b. Inladen en voorbereiden van de surveydata.

Zorg ervoor dat:

- indien met `.sav`-bestanden gewerkt wordt, de variable labels correct zijn zoals gedefinieerd in het codeboek. De functie `SVsurvey_labelled_2_tibble()` kan hierbij gebruikt worden. Deze functie zet de data om naar een nette `tibble` en controleert ook de labels. Via het tweede argument kunnen correcte labels meegegeven worden als `named vector`;
- correcties op basis van open tekstvakken worden doorgevoerd. Hiervoor zet Tina het bestand

```
01_code/03_process_editions/SVXXXX/  
input/SVXXXX_hercodering_obv_tekstvakken.  
xlsx
```

klaar. Deze correcties kunnen doorgaans vlot worden toegepast met een functie zoals `coalesce_join()`;

- indien sportvragen aanwezig zijn, de sportcodering correct wordt verwerkt. Tina documenteert de hercoderingen in:

```
01_code/03_process_editions/SVXXXX/  
input/SVXXXX_hercodering_sport.xlsx
```

op basis van:

```
01_code/03_process_editions/Metadata/  
Sports.xlsx
```

Sporten 1 tot en met 15 worden doorgaans via selectievragen bevraagd en kunnen indien nodig worden gecorrigeerd op basis van open antwoordvakken. Sporten vanaf 16 worden karakterieel opgeslagen in `spt_othx` als een met puntkomma's gescheiden lijst. Zorg er daarbij voor dat steeds de gestandaardiseerde sportnamen uit `Sports.xlsx` worden gebruikt;

- ook hier enkel de variabelen behouden blijven die nodig zijn voor de cleaning;
- ook hier alle variabelen voldoen aan de evaluatiecriteria uit de DSD.

c. **Uitvoeren van de centrale cleaning.**

Laat vervolgens `SVsurvey_clean_data()` lopen op de voorbereide steekproef- en surveydata. Deze functie:

- voert controles uit op consistentie en geldigheid;
- geeft foutmeldingen wanneer de input niet voldoet;
- voert de gestandaardiseerde cleaning uit;
- berekent afgeleide variabelen en gewichten;
- en schrijft de resultaten weg naar:

```
03_deriveddata/05_Cleaned_data  
als .parquet-bestanden.
```

Als er foutmeldingen optreden, kunnen die verschillende oorzaken hebben. Enkele vaak voorkomende situaties zijn:

- **Een waarde in de steekproefdata komt niet voor in een codelijst.**

Dit kan wijzen op een fout in het databestand of op een onvolledigheid in de codelijst. In principe zou het tweede niet mogen voorkomen, maar in de praktijk blijken sommige externe codelijsten niet altijd volledig te zijn. Je kan dan:

- de data manueel corrigeren;
- of de codelijst structureel uitbreiden.

Die laatste optie is duurzamer, op voorwaarde dat je zeker bent dat de code effectief correct en herbruikbaar is.

- **Een waarde in de surveydata komt niet voor in een codelijst.**

Hier kan de oorzaak liggen in:

- het databestand;
- de codelijst;
- of het bestand `Answers.xlsx`.

Ook hier geldt: pas `Answers.xlsx` enkel aan bij echte, bedoelde tekstuele wijzigingen in antwoordverwoording. Gaat het om een fout in de levering door het veldwerkbureau, dan gebeurt de correctie beter lokaal in het cleaning-script.

- **Variabelenamen zijn fout.**

Ook dan zijn er meestal twee opties:

- ofwel corrigeer je de naam tijdelijk in het script;
- ofwel werk je het bestand `SVXXXX_variables_sample.xlsx` of `SVXXXX_variables_survey.xlsx` bij.

Die laatste optie is doorgaans netter en duurzamer.

4.1.4. Aanmaak van de VOS'en

Na de cleaning volgt de aanmaak van de Vlaamse openbare statistieken (VOS'en). Dit zijn de statistieken die gepubliceerd worden op de cijferpagina's en in de cijferapplicatie (Power BI).

Maak hiervoor het script:

```
01_code/03_process_editions/SVXXXX/03_create_VOS_
SVXXXX.R
```

Dit script roept de functie `SVsurvey_create_VOS()` aan.

Deze functie schrijft:

- een analysedataset weg naar:

```
03_deriveddata/06_Analysis_data
```

- en de uiteindelijke VOS-data naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/
999_Public_Use_Files/01_VOSdata/SVsurvey
```

De functie `SVsurvey_create_VOS()` is historisch vrij ad hoc gegroeid. De reden is dat analysevragen gefragmenteerd tot stand kwamen en er lange tijd geen heldere afbakening bestond van wat precies als officiële statistiek moest gelden.

In de cijferpagina's worden vaak cumulatieve proporties voor één specifieke categorie getoond, terwijl in de cijferapplicatie soms percentages per categorie worden weergegeven. Daarom wordt eerst een analysedataset opgebouwd waarin variabelen in de juiste vorm worden klaargezet voor analyse, bijvoorbeeld:

- door variabelen te dichotomiseren;
- door niet-substantiële categorieën te verwijderen;
- door categorische variabelen om te zetten naar numerieke schalen voor gemiddelden.

Deze analysedataset wordt ook weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/021_
SVbevraging/03_deriveddata/06_Analysis_data
```

Alle analysevariabelen zijn opgelijst in:

```
01_code/03_process_editions/Metadata/Variables_
for_VOS.xlsx
```

Na de aanmaak van de analysedataset worden drie typefuncties iteratief toegepast naargelang de kolom type in `Variables_for_VOS.xlsx`:

- berekening van gemiddelden;
- berekening van proporties;
- berekening van multinomiale proporties.

De resulterende VOS'en worden weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_
Public_Use_Files/01_VOSdata/SVsurvey/VOS
```

Daarnaast worden ook item- en unitnonresponsgraden berekend voor gebruik op de cijferpagina's. Deze worden weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_
Public_Use_Files/01_VOSdata/SVsurvey/respons_rates
```

Wat kan beter?

1. De definitie van de officiële statistieken kan scherper worden afgebakend. Een mogelijke keuze is om alle cumulatieve proporties over ordinale antwoordschalen als officiële statistieken te beschouwen. De cijferpagina's zouden dan enkel één geselecteerde categorie tonen, terwijl de cijferapplicatie alle cumulatieve categorieën kan tonen.
2. Een deel van de mapping van bronvariabelen naar analysevariabelen kan mogelijk rechtstreeks worden afgeleid uit mappings die al in de codelijsten aanwezig zijn. In dat geval zou een apart bestand zoals `Variables_for_VOS.xlsx` gedeeltelijk of volledig overbodig kunnen worden.
3. De outputtabellen met VOS'en zijn nog onvoldoende formeel gekoppeld aan het datamodel. Ook de kolomnamen kunnen nog consistentier worden gemaakt.

Opgelet: de data in

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_  
Public_Use_Files/01_VOSdata/SVsurvey/VOS
```

vormt de bron voor de Power BI-applicatie. Als variabelenamen wijzigen, moet dit ook in Power BI worden aangepast.

4.2. KLAARZETTEN VAN TABELLEN VOOR DE CIJFERPAGINA'S

Wanneer de VOS-data beschikbaar is, kunnen daaruit datasets en tabellen worden afgeleid voor de cijferpagina's. Deze tabellen worden nadien rechtstreeks ingeladen in Datawrapper, zodat manuele aanpassingen in principe niet nodig zijn.

Dit gebeurt via het script:

```
01_code/04_publish_tables_cijferpaginas/publish_  
tables_cijferpaginas.R
```

Het script leest automatisch de VOS-cijfers in uit:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_  
Public_Use_Files/01_VOSdata/SVsurvey
```

De tabellen worden aangemaakt op basis van een beperkt aantal grafiek-templates. Een overzicht van alle te genereren tabellen en grafieken staat in:

```
01_code/04_publish_tables_cijferpaginas/tables_
cijferpagina.xlsx
```

Het script leest dit bestand in en maakt alle tabellen of grafieken aan die erin zijn opgelijst. Het bestand bevat onder meer volgende kolommen:

- VOSn: het nummer van de VOS waartoe de grafiek behoort;
- VOSlabel: het label van de VOS waartoe de grafiek behoort;
- graphtype: het type of de template van de grafiek;
- table: de naam van het bestand dat wordt aangemaakt;
- variables: lijst van variabelen die in de tabel opgenomen worden, gescheiden door puntkomma's;
- covariate: het achtergrondkenmerk waarnaar wordt opgesplitst;
- graphtype_description: inhoudelijke beschrijving van de grafiek. Deze kolom wordt momenteel niet gebruikt, maar is wel nuttig als documentatie.

Voor elk graphtype bestaat er in het script een aparte functie. Het script groepeert de rijen per grafiektype en past vervolgens de juiste template-functie toe.

De volgende grafiektypes zijn momenteel gedefinieerd:

- evolutionCI: evolutie van cijfers over de tijd met betrouwbaarheidsintervallen;
- backgroundCI: cijfers voor de laatste editie, opgesplitst naar achtergrondkenmerken, met betrouwbaarheidsintervallen;
- lastobservationCI: cijfers voor de laatste editie, met alle items van een vraag in één tabel;
- evolutioncolumns: evolutie in tabelvorm met tijdsdimensie in kolommen;
- lastobservationbycovarcolumns: cijfers voor de laatste editie, uitgesplitst naar een achtergrondkenmerk, met achtergrondkenmerken in kolommen;
- lastobservationcolumns: cijfers voor de laatste editie, met alle antwoordcategorieën in kolommen;
- top10bar: cijfers voor de laatste editie, beperkt tot de top 10 categorieën.

Wanneer een nieuwe grafiek past binnen een bestaande template, volstaat het om een extra rij toe te voegen aan `tables_cijferpagina.xlsx`. Wan-

neer een nieuw type visualisatie nodig is, moet daarvoor een nieuwe functie worden toegevoegd aan het script.

De tabellen – en sporadisch ook statische grafieken – worden weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_
Public_Use_Files/02_CPdata
```

Wat kan beter?

Momenteel worden de tabellen voor de cijferpagina's telkens opnieuw aangemaakt voor alle VOS'en, ook wanneer die in de laatste editie niet werden bevraagd. Dat is niet efficiënt. Op langere termijn wordt dit best omgezet naar een editie-specifieke functie binnen:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/021_
SVbevraging/01_code/03_process_editions
```

zodat enkel de tabellen worden opgebouwd voor de VOS'en die daadwerkelijk in de betrokken editie voorkomen.

4.3. CREATIE VAN HANDLEIDING EN TECHNISCHE NOTA

De volgende stap is de aanmaak van de handleiding en de technische nota. Dit gebeurt via het script:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/
021_SVbevraging/01_code/05_Create_manual_and_
technical_note/create_documentation.R
```

De handleiding en de technische nota worden gecompileerd met $\text{\LaTeX}$. Daarvoor moet een $\text{\LaTeX}$ -distributie geïnstalleerd zijn op de computer. Annelies en ik gebruiken MiKTeX:

<https://miktex.org/howto/install-miktex>

Dit programma is niet beschikbaar via het bedrijfsportaal en moet dus via IT worden geïnstalleerd.

De inhoud van de handleiding en technische nota vertrekt vanuit het .tex-bestand:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/
021_SVbevraging/01_code/05_Create_manual_and_
technical_note/Input/Handleiding_SV-bevragingen.
tex
```


In dit bestand kunnen tekstuele aanpassingen worden gemaakt. Het document bevat macro's waarmee onderscheid kan worden gemaakt tussen:

- delen die enkel in de handleiding terechtkomen;
- en delen die zowel in de handleiding als in de technische nota worden opgenomen.

Daarnaast voegt het R-script automatisch tabellen in, bijvoorbeeld met een overzicht van de vragen of van de responsgraden. Het R-script compileert ook automatisch de $\text{\LaTeX}$ -bestanden.

De handleiding en technische nota worden weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_
Public_Use_Files/04_microdata
```

Wat kan beter?

De inhoud van de handleiding en de technische nota overlapt momenteel sterk. Op termijn is het wenselijk om duidelijker af te spreken:

- welke informatie intern in de handleiding thuishoort;
- welke informatie extern in de technische nota gepubliceerd wordt;
- en welke onderdelen best automatisch uit metadata en scripts worden gegenereerd.

4.4. CREATIE VAN PUBLIC EN SCIENTIFIC USE FILES

De laatste stap in het proces is de aanmaak van de Public Use File (PUF) en de Scientific Use File (SUF) voor de microdata. Dit gebeurt via het script:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/021_
SVbevraging/01_code/06_Publish_PUF_SUF/Publish_
PUF_SUF.R
```

Dit script maakt gebruik van het bestand:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/
021_SVbevraging/01_code/06_Publish_PUF_SUF/Input/
Variables_Use_files.xlsx
```

Dit bestand bevat een overzicht van alle variabelen die in beide use files mogen worden opgenomen. Het bevat minstens volgende kolommen:

- `concept_id`: de naam van de variabele;
- `ScientificUseFile`: bevat een x wanneer de variabele opgenomen wordt in de Scientific Use File;

- `PublicUseFile`: bevat een `x` wanneer de variabele opgenomen wordt in de Public Use File.

Het R-script leest dit bestand in, selecteert op basis daarvan de juiste variabelen uit de gekuiste data en schrijft de bestanden weg als `.sav`-bestanden. Daarbij worden de juiste variabelelabels en value labels toegevoegd, zodat gebruikers een bruikbaar en net bestand krijgen.

Als variabelen moeten worden toegevoegd of verwijderd uit de use files, volstaat het dus om `Variables_Use_files.xlsx` aan te passen. De scripts zelf hoeven daarvoor normaal niet gewijzigd te worden.

De Scientific Use File wordt weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/998_Scientific_Use_Files/021_SVbevraging
```

De Public Use File wordt weggeschreven naar:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/999_Public_Use_Files/04_microdata
```

Wat kan beter?

Momenteel wordt steeds één grote public use file en één grote scientific use file aangemaakt waarin alle edities van de SV-bevraging zijn samengevoegd. Die keuze werd destijds gemaakt omdat collega's moeilijkheden hadden om edities zelf correct te koppelen.

Op termijn is deze aanpak echter niet houdbaar, omdat de bestanden steeds groter worden. Het is daarom beter om per editie een afzonderlijke PUF en SUF aan te maken. Ook dit script wordt dus bij voorkeur omgezet naar editie-specifieke functionaliteit binnen:

```
//WV162699/fs_kb_dkb/svr/Beveiligde_data_VSA/021_SVbevraging/01_code/03_process_editions
```

zodat enkel de bestanden voor de betrokken editie worden gegenereerd.

5. NIEUWE VRAGEN

Wanneer er een nieuwe vraag wordt toegevoegd aan de SV-bevraging, moet een reeks opeenvolgende stappen worden doorlopen die het volledige verwerkingsproces bestrijken: van het datamodel tot de publicatie.

Doorloop deze stappen in de aangegeven volgorde, omdat elke stap voortbouwt op de vorige.

1. Maak of actualiseer de codelijst in het datamodel.

Als de nieuwe vraag antwoordcategorieën bevat die nog niet voorkomen in de bestaande codelijsten, moet een nieuwe codelijst worden aangemaakt of een bestaande codelijst worden uitgebreid.

Controleer eerst of de benodigde waarden al voorkomen in:

```
01_code/01_data_model/01_code_lists/01_
general_codelists/input/code_list_values.xlsx
```

Voeg ontbrekende waarden daar toe. Definieer vervolgens de structuur van de nieuwe codelijst in:

```
input/code_list_skeletons.xlsx
```

Voer daarna `create_code_lists.R` opnieuw uit zodat de codelijst wordt aangemaakt in:

```
03_deriveddata/01_codelists
```

Zie paragraaf 2.1 voor meer uitleg.

2. Voeg een nieuw concept toe aan de DSD.

Voeg de nieuwe variabele toe aan:

```
01_code/01_data_model/02_DSD/dsd_variables.
xlsx
```

Vul daarbij alle relevante kolommen correct in:

- `concept_id`;
- beschrijving;
- labels in de beschikbare talen;
- datatype;
- codelijst, indien van toepassing;
- eventuele validatieregels.

Voer daarna `check_and_publish_dsd.R` uit. Dit script valideert de invoer en schrijft de bijgewerkte DSD weg naar:

```
03_deriveddata/02_DSD/dsd.xlsx
```

Werk pas verder wanneer deze stap zonder foutmeldingen is doorlopen.

3. Registreer de vraagverwoording in `Questions.xlsx`.

Voeg een rij toe aan:

```
01_code/03_process_editions/Metadata/Questions.
xlsx
```

Koppel de vraagverwoording aan het juiste `concept_id` en vul minstens de kolommen `question`, `question_item`, `question_module` en `from_surv_id` correct in.

Zie paragraaf 4.1.2 voor meer uitleg.

4. **Registreer de antwoordverwoordingen in `Answers.xlsx`.**

Voeg de relevante antwoordverwoordingen toe aan:

```
01_code/03_process_editions/Metadata/Answers.
xlsx
```

Koppel elke antwoordverwoording aan de juiste `value` uit de code-lijst en vul opnieuw `from_surv_id` in.

Zie paragraaf 4.1.3 voor meer uitleg.

5. **Neem de variabele op in de cleaning.**

Voeg de nieuwe variabele toe aan:

```
01_code/03_process_editions/SVXXXX/input/
SVXXXX_variables_survey.xlsx
```

Vul in kolom `variable` de naam in zoals het veldwerkbureau die aanlevert, en in kolom `concept_id` de gestandaardiseerde naam uit de DSD.

Pas vervolgens indien nodig ook het script

```
01_code/03_process_editions/SVXXXX/02_clean_
data_SVXXXX.R
```

aan, bijvoorbeeld wanneer bijkomende correcties op open tekstvakken nodig zijn.

Zie paragraaf 4.1.3.

6. **Werk de centrale cleaningfunctie bij indien nodig.**

Wanneer de nieuwe variabele een specifieke bewerking vereist tijdens de geautomatiseerde cleaning – bijvoorbeeld een afgeleide variabele, een specifieke hercodering of een niet-standaard validatieregel – dan moet dit worden toegevoegd aan:

```
01_code/00_functions/R
```

meer specifiek in de functie `SVsurvey_clean_data()`.

Documenteer zulke wijzigingen via Roxygen2 in de code zelf. Wees voorzichtig met wijzigingen in deze functie, omdat ze impact hebben op alle edities.

7. **Neem de variabele op in de VOS-aanmaak.**

Voeg de nieuwe analysevariabele(n) toe aan:

```
01_code/03_process_editions/Metadata/  
Variables_for_VOS.xlsx
```

Definieer:

- de naam van de analysevariabele;
- het relevante VOS-nummer en VOS-label;
- het type statistiek (percentage, multinom of mean);
- en een bruikbare beschrijving.

Pas ook het script

```
01_code/03_process_editions/SVXXXX/03_create_  
VOS_SVXXXX.R
```

aan wanneer de analysevariabele nog expliciet moet worden aangemaakt in de analysedataset.

Zie paragraaf 4.1.4.

8. **Neem de variabele op in de PUF en/of SUF.**

Voeg de variabele toe aan:

```
01_code/06_Publish_PUF_SUF/Input/Variables_  
Use_files.xlsx
```

Zet een x in de kolom ScientificUseFile en/of PublicUseFile naargelang de variabele mag worden opgenomen in de relevante use file.

Wanneer er twijfel bestaat over de privacygevoeligheid van een variabele, overleg dan eerst met de data manager of DPO voordat de variabele in een PUF wordt opgenomen.

9. **Voorzie tabellen of grafieken voor de cijferpagina's.**

Voeg indien nodig één of meerdere rijen toe aan:

```
01_code/04_publish_tables_cijferpaginas/  
tables_cijferpagina.xlsx
```

Gebruik een bestaand graphtype wanneer de gewenste visualisatie al ondersteund wordt. Als een nieuw type visualisatie nodig is, moet daarvoor een nieuwe functie worden toegevoegd aan:

```
publish_tables_cijferpaginas.R
```

Zie paragraaf 4.2 voor een overzicht van de beschikbare grafiektypes.